ARENAFIGHT AI RUNTIME

System Architecture, Physical Evidence Verification & Autonomous Multi-Model Engine

Version: 2.5.0 Enterprise CLI | **Release:** Production | **Target Platforms:** Windows (Win32/x64) & Linux (POSIX)
Author: Lunarmist-byte | **GitHub:** https://github.com/Lunarmist-byte | **LinkedIn:** https://www.linkedin.com/in/amal-s-kumar-ba69a1290/

1. Executive Summary & Design Philosophy

ArenaFight is a high-performance, deterministic C++20 multi-model autonomous agent runtime designed to completely eliminate hallucinated success in artificial intelligence workflows. While conventional agentic systems rely upon an LLM self-reporting whether its code or solution functions, ArenaFight enforces the **Level 1 Physical Dominance Rule**: an assertion is classified as ground truth only when corroborated by concrete operating system artifacts, process return codes, compiler diagnostics, and cryptographic hashes.

The runtime operates across dual paradigms: functioning either as a blistering fast, zero-overhead **Standalone Native CLI** with direct hardware and local provider hooks, or as a synchronized **OpenCode Connected Bridge**. It autonomously assesses host compute topologies, manages background AI service daemons, and enforces rigorous defense-in-depth isolation.

2. What ArenaFight Does Well: Architectural Deep-Dive

A. Level 1 Physical Dominance Rule & 6 Physical Verification Gates

Every code mutation, task execution step, and generated artifact passes through an uncompromising 6-gate physical verification harness. Unverified assertions are strictly discarded by the ClaimRegistry.

Gate #	Verification Type	Physical Evidence Sensor	Pass Criteria & Deterministic Latch
Gate 1	Compiler & Build Diagnostics	Direct OS process exit code + stderr pipe from g++, clang++, or cl.exe	Zero compile errors, zero fatal diagnostics; clean binary artifact creation
Gate 2	Automated Test Suite Execution	Test runner stdout stream, assertion counters, process exit status	Process exit code 0; 100% assertions green; zero test failures or leaks
Gate 3	Process Execution Validation	OS PID spawn, stdout/stderr capture, memory/cpu heartbeat	Process terminates with return code 0 or matches expected output contract
Gate 4	Cryptographic Hash Integrity	FIPS 180-4 SHA-256 digest computation over generated disk artifacts	Hash matches manifest; audit log record appended with parent hash pointer
Gate 5	Filesystem Mutation Audit	Pre/post execution directory diff and inode/timestamp tracking	Target files created/modified within path boundaries; zero unauthorized leaks
Gate 6	Quality of Service (QoS)	Static AST scanner inspecting heap allocation and blocking calls	Zero-heap verification in ISR/tight-loops; deterministic latency bounded

B. Dual Processing Levels: Fast vs Perfect Boi

ArenaFight eliminates the one-size-fits-all tradeoff between turnaround speed and absolute correctness by introducing two specialized runtime execution levels (switchable via /level [fast|perfect]):

- **FAST Level:** Engineered for rapid iteration, daily coding tasks, and low-latency interaction. Employs standard reasoning token budgets, fast path verification, and rapid single-turn self-correction.
- **PERFECT BOI Level:** Deep, exhaustive perfectionist execution mode. ArenaFight dynamically scales its reasoning depth, adversarial cross-checks, and execution iterations up to the absolute physical capability of the host machine. No stone is left unturned: every gate must pass with cryptographic proof, and solutions are adversarially stress-tested against synthetic edge cases.

C. Dynamic Host PC Hardware Profiling (Zero Hardcoded Limits)

Unlike brittle agents that hardcode token ceilings, timeouts, or thread limits, ArenaFight features an autonomous host machine profiler (SystemProfiler). It actively inspects the system topology on Windows and Linux:

- **RAM Inspection:** Detects installed physical RAM and dynamically computes free available memory.
- **CPU Core Topology:** Measures physical cores and logical hyper-threads to set parallel execution limits.
- **Dynamic Budget Derivation:** In Perfect Boi mode, limits scale dynamically:
  * High-end workstations (>32GB RAM, >12 cores): 60 iterations, 32,768 reasoning tokens, 4 adversarial passes.
  * Mid-range PCs (16-32GB RAM, 8-12 cores): 35 iterations, 16,384 tokens, 3 adversarial passes.
  * Constrained laptops (<16GB RAM): safe, un-choked budgets scaled down to preserve host responsiveness.

D. NVIDIA CUDA Hardware Acceleration & Graceful Slow-Variant Fallback

ArenaFight natively probes for dedicated NVIDIA graphics processing units via driver APIs and nvidia-smi. When an NVIDIA GPU is present, tensor acceleration, VRAM allocation, and local model inferencing are routed through CUDA.

Explicit Slow-Variant Classification: If the host lacks a dedicated CUDA GPU (operating on an integrated GPU or pure CPU), ArenaFight refuses to silently degrade or freeze. Instead, it explicitly classifies the environment as a **Slow-Variant Runtime**, surfaces clear latency expectations in the terminal dashboard, and throttles parallel batch sizes to prevent memory starvation and thrashing.

E. Autonomous Background Service Daemons (Ollama Zero-Friction)

ArenaFight eliminates the friction of requiring users to manually launch terminal servers or AI daemons. The ServiceDaemon subsystem provides autonomous daemon lifecycle management:

- **Autonomous Discovery:** Scans standard binary distributions on Windows (%LOCALAPPDATA%\Programs\Ollama) and Linux (/usr/local/bin, /usr/bin).
- **Process-Level Duplication Prevention:** Queries OS process snapshots via Win32 CreateToolhelp32Snapshot and Linux pgrep before attempting to spawn, ensuring no duplicate instances are created.
- **Thread-Safe Mutual Exclusion & Cooldown Latch:** A thread-safe mutex and 45-second launch cooldown latch completely eliminate background process spawning storms.
- **Fast-Path Loopback Polling:** Direct IPv4 loopback (127.0.0.1) avoids IPv6 [::1] name resolution delays on modern Windows builds, achieving sub-millisecond status responsiveness.

F. Universal Model Benchmarking & Thinking Phase Duels

The CLI includes a universal benchmarking engine capable of evaluating any local or remote model across four distinct vectors: **Reasoning**, **Code Generation**, **Physical Verification**, and **Tokens-per-Second Throughput**. Furthermore, the /think (alias /duel) tournament command disallows code generation entirely, pitting competing AI models against each other to evaluate pure chain-of-thought logic, architectural reasoning, and trade-off synthesis.

G. Hardware Studio & Embedded QoS Engineering

ArenaFight incorporates a specialized embedded systems studio catering to mission-critical hardware firmware development:

- **Board & Pinout Intelligence:** Complete hardware registry for ESP32, ESP32-S3, RP2040, STM32F4, and Arduino boards, tracking flash sizes, SRAM, strapping pins, ADC channels, and default bus configurations.
- **Static QoS Compliance Engine:** Enforces zero-heap allocation inside interrupt service routines (ISRs) and disallows blocking delays (delay()) in real-time loops.
- **Circuit Design Calculators:** Built-in analytical solvers for LED current-limiting resistors, analog voltage dividers, I2C pullup resistors, and battery discharge lifespans.
- **Wokwi Simulation Scaffolding:** Generates instant diagram.json circuit simulation files alongside firmware.

H. Defense-in-Depth Security Sandbox

To safeguard the developer host from rogue scripts or accidental credential leakage, ArenaFight enforces three security tiers (Strict, Balanced, Permissive):

- **Path Confinement (Jail):** Rejects any file access, read, or write operation that resolves outside the authorized workspace directory.
- **Data Loss Prevention (DLP):** Scans workspace files and agent outputs for sensitive tokens, AWS keys, GitHub tokens, and private RSA keys.
- **SHA-256 Tamper-Evident Audit Chain:** Every security event, tool execution, and configuration change is hashed and cryptographically linked to the previous log record, creating an unalterable blockchain-style verification trail.

3. Technical Architecture Specification

Subsystem	Implementation Details	Key Capabilities
Core Language & Standard	C++20 (GCC 14 / Clang 18 / MSVC 19.3x)	Concepts, std::span, std::string_view, filesystem, zero-copy string manipulation
CLI REPL Engine	ReplEngine with unified command dispatcher	Interactive slash commands, context mentions (@file, @git, @claims, @memory), live telemetry
Provider Bridge	Modular multi-provider routing layer	Ollama (local), OpenCode CLI, OpenRouter, Claude Code, custom OpenAI-compatible endpoints
Hardware Profiler	SystemProfiler (Win32 API & POSIX /proc)	Probes RAM, CPU core topology, NVIDIA CUDA GPU state; computes non-hardcoded dynamic limits
Service Daemon	ServiceDaemon background process manager	Auto-detects & starts Ollama, process-level deduplication, cooldown latches, model puller
Telemetry Observatory	TerminalUI high-density ANSI terminal	Real-time latch counters, memory pressure gauges, gate verification pass rates, latency metrics
Security Sandbox	SecuritySandbox & Audit Trail	Workspace path jail, DLP secret scanner, FIPS 180-4 SHA-256 cryptographic audit chain

4. Categorized Slash Command Quick-Reference

ArenaFight v2.5.0 features a completely restructured, de-duplicated command reference. Aliases are documented inline next to primary commands across 6 logical functional groups:

Category & Command	Inline Alias	Functional Description & Usage
1. Autonomous Execution	-	Autonomous self-healing goal pursuit (/goal <task>)
/goal	-	Milestone plan generator (/plan <task>)
/plan	-	Interactive requirements interview (/grill-me <topic>)
/grill-me	**/duel**	Thinking tournament: Compare reasoning without code gen
/think		
2. Hardware & Processing	**/processing**	Set processing level: Fast (standard) vs Perfect Boi (adaptive)
/level	-	Assess physical host PC hardware & dynamic limits (/profile [cpu|cuda])
/profile	**/daemon**	Autonomous service daemon manager: auto-detect and start Ollama
/services		
3. AI Models & Providers	**/model**	List discovered models or set active model (/models set <name>)
/models	-	Display multi-dimensional capability breakdown matrix
/abilities	**/bench**	Universal benchmark across reasoning, code, verification & throughput
/benchmark	-	Switch mode (standalone native vs OpenCode connected)
/mode	-	Connect and synchronize with OpenCode bridge
/connect	-	OpenCode model discovery, execution and status
/opencode	-	Unsloth AI on-the-fly fine-tuning engine
/unsloth		
4. Security & Telemetry	-	View runtime execution mode, session state & verification stats
/status	**/telemetry**	Display terminal live telemetry observatory & gate pass rates
/metrics	-	Inspect defense-in-depth posture, scan workspace, set level
/security	-	Inspect tamper-evident SHA-256 audit log and verify hash chain
/audit		
5. Embedded Studio	**/board**	List or inspect microcontroller boards, pinouts, strapping pins
/boards	-	List supported sensors, actuators, and communication ICs
/components	-	Scaffold embedded project with Wokwi diagram and QoS firmware
/scaffold	-	Evaluate firmware for zero-heap, non-blocking QoS compliance
/qos	-	Calculate circuit values: LED, divider, I2C pullup, battery life
/calculate		
6. Context & Utilities	**/file**	Inspect, view, read, and attach workspace files to prompt
/files	-	Persist Knowledge Items (KI) or rules into repo memory
/learn	-	Schedule periodic or delayed autonomous task execution
/schedule	-	Inspect and preview generated project artifacts
/artifacts	-	Manage agent skills and system/workspace operational rules
/skills, /rules	**/temperature**,	Adjust temperature (0.0-2.0) and max generation token limits
/temp, /tokens	**/max-tokens**	Interactive guided help, screen clear, and REPL exit
/guide, /clear, /exit	**/help-session**, **/quit**	

5. Interactive Context Mentions

Context Mention	Runtime Syntax	Context Expansion & Injection Mechanism
File Injection	`@file <path>`	Reads the physical file from disk, attaches it to the session, and injects contents directly into prompt context.
Working Tree Diff	`@git`	Executes git status and git diff; injects unstaged/staged working tree changes into prompt.
Verified Claims	`@claims`	Injects ClaimRegistry evidence report and physically verified facts discovered during the session.
Session Memory	`@memory`	Injects historical session discoveries, user preferences, and persistent repository Knowledge Items.

6. Project Attributions & Licensing

ArenaFight is authored and engineered by **Lunarmist-byte**. Licensed under custom All Rights Received terms: strictly free for personal non-commercial experimentation and research; commercial and enterprise deployment requires written licensing authorization.

Repository & Developer Links:
- **Author:** Lunarmist-byte
- **GitHub Repository:** https://github.com/Lunarmist-byte
- **LinkedIn Profile:** https://www.linkedin.com/in/amal-s-kumar-ba69a1290/